

Overview of parallel processing *

G.S. ALMASI

Thomas J. Watson Research Center, Yorktown Heights, NY 10598, U.S.A.

Received June 1985

1. Introduction

We are on the threshold of a new era in computer architecture. It is becoming increasingly difficult to obtain more performance from the time-honored von Neumann model, and many of the technological constraints that influenced its design over thirty years ago have changed drastically. Many of the arguments for processing a single instruction at a time no longer apply, and a number of enthusiastic parallel processing projects are working on various ways to allow many processors to work on a single problem at the same time. However, this re-opens a Pandora's box of questions about how computation should be done, and some of the strengths of the von Neumann model which temporarily closed this box three decades ago become especially apparent when one tries to replace it. This overview treats the promises and accomplishments of parallel processing as well as the problems and work that remain. The paper is organized as follows:

- Current driving forces for parallel processing;
- Definitions and fundamental questions;
- Survey of projects;
- Emerging answers.

As will be shown, the field is at an interesting juncture. Much work has been done, and the ideas are now there for putting it all together. But some large experiments are needed to provide real results from real programs if the pace of progress is to be maintained.

2. Driving forces

Three prominent driving forces for parallel processing are:

- (1) performance,
- (2) cost-performance,
- (3) programmer productivity.

These are influenced by both hardware and software. Hardware technology (chiefly VLSI) has led to a repeal of 'Grosch's law', a long-valid observation that the best price/performance was obtained with the most powerful uniprocessor. Thus it is no longer automatically true that a collection of smaller processors will always have less performance than a single large processor of the same cost. At the same time, physical effects such as the finite speed of light are making it increasingly difficult for technology alone to increase the speed of the fastest serial processors. As a consequence, a parallel computer made up of a number of small cooperating processing

* Published with permission of the Benjamin-Cummings Publishing Co., which will include portions in an upcoming book.

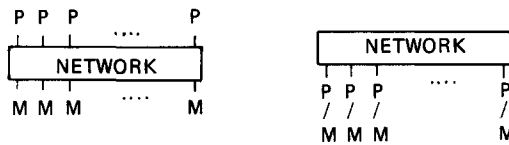


Fig. 1. Generalized Parallel Processor: The left side shows the 'dancehall' configuration, with processing elements on one side of a switching network and memory elements on the other. In the "boudoir" arrangement on the right, each processing element has its private memory, and these pairs of elements are connected to the network. Dancehall arrangements are often associated with a shared-memory computational model, while boudoir configurations are often used with a message-passing model.

elements becomes an attractive possibility for the scientist or engineer whose problem is too large or whose budget is too small for today's supercomputers.

From the software side, there is a strong argument that programmer productivity is declining and that new languages and new architectures to support them are needed. John Backus, the inventor of FORTRAN, has recently characterized all such von Neumann languages as 'fat and flabby' because they are at too low a level and force the programmer to spend too much time on *how* the program will run on the von Neumann architecture, when the programmer's time should really be spent on describing *what* is to be accomplished by the program. Backus' solution [1] is a higher-level 'function-level programming' language, discussed in this issue by Kogge [12]. One of its features is that it is a 'single-assignment' language; this eliminates 'side effects' and makes parallel execution of programs easier, but efficient execution requires architectures without addressing, such as those based on dataflow or reduction. Some information and references on these computational models can be found in the abstract in this issue by Treleven [20].

3. Definitions and fundamental questions

The following definition covers a large number of highly parallel processor designs (also see Fig. 1):

"A collection of serial processors¹ which can communicate and cooperate to solve big problems fast."

However, we shall soon see that this raises more questions than it answers.

Two terms, SIMD and MIMD, are frequently used in discussing parallel processors. These come from a classification originally made by Flynn [5], still widely used and summarized in Table 1.

Now let's return to our definition of a parallel processor and list some of the questions it raises:

- (1) "*A collection of serial processors...*"
 - How many?
 - How big?
 - Architecture, instruction set?
 - How much memory?
 - How much I/O?

¹ This definition leaves out computers with multiple functional units (multipliers, etc.) running in parallel. These are also interesting but are not treated here due to lack of space.

Table 1

Instruc. stream	Data stream	
	Single	Multiple
Single	SISD (von Neumann)	SIMD (vector, array)
Multiple	MISD (pipe- line?)	MIMD (multiple micros)

- (2) "...which can communicate..."
 - Interconnection network?
 - Means of communication?
 - Means of synchronization? (See Fredrickson et al. [7].)
- (3) "...and cooperate..."
 - Grain size?
 - SIMD/MIMD?
 - Operating System? (See Browne [3].)
- (4) "...to solve big problems fast."
 - Which problems? (Which applications?)
 - Computational model?
 - Programmer's view?
 - How is a problem decomposed?
 - How is concurrent execution specified?
 - Who does each of the above?
 - Which language? new features? (See the papers by Ghezzi [8] and Kogge [12].)
 - Compiler?
 - Algorithm?
 - How much speedup?
 - Reliability, Availability, Serviceability?

Item (1) can be thought of as *architecture*-related, item (4) as *software*-related, while items (2) and (3) fall under both umbrellas.

4. Survey of projects

Some of the answers to the questions posed above are provided by surveying recent parallel processor products and research projects. This is done in Tables 2–4 and Fig. 2.

A note first. There is always a danger in publishing a survey like this one. It is by necessity nonexhaustive, and reflects this author's interests. Thus it lists chiefly U.S. projects, and deals with levels of parallelism above that of a 2-way or 4-way multiprocessor. The purpose is to provide some of the answers we are seeking, not to publish a complete catalogue of projects.

This survey was originally inspired by a shorter taxonomy presented by J. Schwartz at the 1982 International Conference on Parallel Processing. He grouped projects chiefly by two characteristics:

- Processor size (small/substantial);
- Interconnection network (packet/circuit switched).

Our own extended version of this survey appears in Tables 2–4 and in Fig. 2, which is a graphical representation of those tables.

Table 2
Parallel processor survey (Small PE's)

Project	Application	Architecture	Prototypes	Notes
MIT CONNECTION Memory/Machine	AI (Knowledge base manip., vision)	10 * 6 PE's (bit-sr1), n-cube & mesh conn., message passing	64K-256K PEs by '85? 2 chip types	1 node of semantic network per PE; SIMD; 32 PE/chip?
Columbia U. NON-VON	Database, AI,	10 * 6 PE's (1-8 bit) tree & omega conn., message passing	To be defined; 1 chip type	1 record/PE; SIMD, MSIMD? 16 PE/chip?
N. Carolina U. CELLULAR Computer	Direct exec. of Backus' FP language	10 * 6 PE's? tree connected string reduction	15 PE 2 chip types	1 FP symbol per leaf cell MSIMD
Goodyear MPP (Massively Parall. Proc.)	Satellite image processing	16K PE's (128 x 128 array)	Delivered to NASA	SIMD 8 PE/chip (CMOS/SOS)
ICL DAP (Distributed Array Proc.)	Matrices partial diff. eqns.	4K PE's (1-bit) (64 x 64 mesh)	Product	SIMD Has DAP-FORTRAN
CMU SYSTOLIC Arrays	Algorithms like matrix multiplicat.	Collection of special purpose units	Various VLSI chip	Spec. purp. VLSI implementations of algorithms

5. Emerging answers

The information in the survey provides some of the answers to the questions raised in Section 3. The order below is the same as in that section.

(1) "*A collection of serial processors...*"

- How many? How big? What kind?

The projects tend to cluster into three groups, those with a number of processors on the order of ten, on the order of thousands, and on the order of a million.

The processors in the first group are *multi-chip*, and correspond to the philosophy of a parallel computer built from a few very fast sequential processors. Processors in the second group are usually *single-chip*, and represent an attempt to take advantage of available microprocessors. Processors in the third group are usually *many-per-chip* (usually bit serial), and usually represent custom VLSI designs.

- Memory and I/O?

Only some general comments are provided in answer to this important question. The memory and I/O structure will be influenced by at least three factors:

- (a) The power of the processors;
- (b) The needed ratio of processing to communication;
- (c) The ratio of short to long data moves (locality).

The last two items especially will depend strongly on the application, and can be expected to be quite different for problems in numeric computation, symbolic computation, and database operation.

(2) "*...which can communicate...*"

- Interconnection network?

Five categories of interconnection network appear in the survey. They are listed below along with the projects using them:

- (a) Crossbars

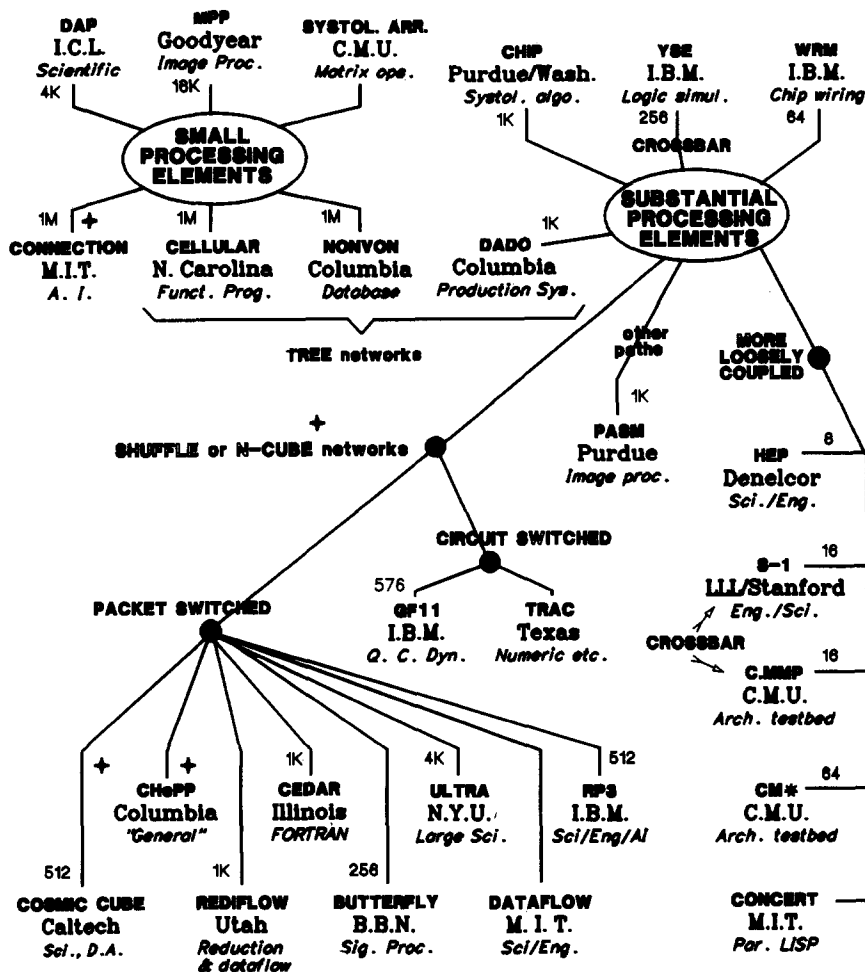


Fig. 2. An overview of parallel processors: The grouping is chiefly by the size of the processing elements and the method of interconnection. The application and the number of processing elements is also shown ($K = 1024$, $M = K \times K$). (After Schwartz [19]). See Tables 2–4 for more details.

- (C.MMP, S-1, YSE)
- (b) Banyans, omegas, shuffles, ...
 - (ULTRA, TRAC, CEDAR, BUTTERFLY, REDIFLOW, DATAFLOW, RP3, GF11).
- (c) Cubes
 - (Cosmic Cube, Connection Machine, CHoPP).
- (d) Trees
 - (NON-VON, DADO, Cellular Computer).
- (e) Arrays
 - (CHiP, DAP, Illiac IV, MPP, Systolic Arrays, WRM).

These represent a sequence of tradeoffs between performance (communication bandwidth and delay) and cost (network size and scaling).

The crossbar allows any processor to talk to any other idle processor or memory at any time, with a delay of one switching element. It requires the most hardware for a given number of processors, and scales as the square of the number of processors.

Shuffle-exchange networks are a lower-cost, lower-performance alternative. A processor

Table 3
Parallel processor survey (Large PE's, packet switched)

Project	Application	Architecture	Prototypes	Notes
NYU ULTRA-computer	Large scientific problems	4K microprocs. (32b) omega-net connection Shared memory	8-PE '84; 64-PE next	Fetch & add, combng. 10Gflops (1990?) MIMD; packet sw.
U. Illinois CEDAR	Scientific, engineering (FORTRAN)	1K PE's, omega net, shared mem. & global dataflow	32-PE, 10 Mflops ea., '87	MIMD, circuit switched, 'Paraphrase' compiler
Columbia U. DADO	AI (Knowledge based/Export systems)	1K custom PE's tree-connected	16-PE '83	Production machine (Facts & rules) MIMD
Univ. of Utah REDIFLOW	AI (Knowledge based/Expert systems)	1K 'Xputers' (32 × 32 & shuffle) reduction & dataflow		Graph reduction PROLOG, Funct. Lang. MIMD
IBM RP3 (Res. Parallel Proc. Prototype)	Large sci., eng., AI, graphics	512 microproc. (32b) partitionable local/global mem.; cache	16-PE, '85	MIMD; can optimize for shared memory or message passing
Caltech COSMIC CUBE	Scientific, Design Automation	512 PE's (16-32b) N-cube connection homogeneous machine	64-PE, '83	User (re)writes algorithms for mach. MIMD, Msg. passing
BBN (Bolt, Bernk., Newmn.) BUTTERFLY	Real-time Voice data mpx-dempx	256 PE's (68000-bsd) Shuffle network homogeneous machine	16-PE, '82	Global memory provision MIMD
Columbia CHoPP (Homog. Par. Processor)	'General purpose'	N-cube network	Sullivan Computer Inc.	MIMD; Conventional language
MIT DATAFLOW (Dynamic)	Scientific, engineering	'Tagged-token' dataflow	Emulator: 64 LISP machs. ('85?)	Multiple tokens allowed per arc; MIMD
MIT DATAFLOW (Static)	Scientific, engineering, sig. proc.	dataflow	6800-based, working	Only one token allowed per arc; MIMD

can talk to any other processor or memory *most* of the time if things are right. Its size scales as $N \log N$, where N is the number of processors, and the communication delay is proportional to $\log N$ switch cycles.

In a ' $\log N$ ' cube network, processors and memories are paired, and each pair is connected to $\log N$ nearest neighbors. Communication with any other pairs must be relayed via these neighbors. The size and delay scale in the same way as a shuffle network. Cube networks are used in both SIMD and MIMD architectures, and represent a sort of boundary between them: crossbars and shuffles appear mostly in MIMD designs, trees and arrays mostly in SIMD.

Tree and array networks also consist of interconnected processor-memory pairs, but with only three or four nearest neighbors. Tree networks are good for sorting operations. The communication delay scales as $\log N$. A potential problem is congestion at the root (vertex). Arrays are used successfully for image processing and for

Table 4
Parallel processor survey (Large PE's, mostly circuit switched)

Project	Application	Architecture	Prototypes	Notes
MIT CONCERT Myriaprocessor	Parallel LISP	10 * 4 PE's, block-oriented shared memory	"CONCERT" = 3(?) 68000's, '84	MIMD, circuit switched, no caches
U. Texas TRAC (Tex. Reconfig. Array Comput.)	Numeric, image proc., database	Banyan network, reconfigurable	4-PE, 7 mem '82	MIMD, circuit switched, workbench, operating sys. work
Purdue/U. Wash. CHiP (Config. Hi. Par. Mach.)	Systolic algorithms, spec. purp.	1K PE's switchable systolic arrays		MIMD An approach to wafer scale integration
Purdue PASM (PARallel SIMD/ MIMD Machine)	Image proc., speech, parallel LISP	1K PE's (32 groups of 32 cube-connected PE's)	16 68000's, '84	SIMD/MIMD 1 microcontroller per 32 PE's
IBM GF-11	Quantum chromo- dynamics	576 custom PE's, Benes network of 24 x 24 crossbars	operational in '85 (?)	SIMD, conditional execution and relocation
IBM YSE (Yorktown simu- lation engine)	Logic simulation	256 PE's (custom) crossbar	16-PE, '82 connected	MIMD, 4K logic gates/PE, high speed simulation
IBM WRM (Wire routing machine)	Gate array (masterslice) wiring	64 Z80's (8 x 8 mesh) Master/slave	operational in '80	MIMD wired 400-gate chips
Yale ELI (Extra Long word Instructions)	Sci. code w. many loops, few branches	Central control, vector-like, (512-bit instrucs.)	16-PE (Built after good code gen.)	SIMD ordinary languages mult. funct. units
CMU/DEC Parallel Proc. Workbench	Parallel Processing Testbed	64 PE's (VAX) Shared memory (single bus)	Not built yet	MIMD, packet switch. cache coherence solved using bus
CMU CM*	Operating system testbed	50 PE's (DEC LSI-11) Cluster, controller, 4 clust., bus conn.	operational in '80	MIMD, packet switch. Considerable work on Operating Systems
CMU C.MMP	Architecture testbed	16PE's, 16 mems., crossbar connected	operational in '80	MIMD, circuit switched
LLL/Stanford S-1		16 PE's	operational in '82 (1 PE)	MIMD
Denelcor HEP (Heterogeneous Elem. Proc.)	Scientific, engineering	8 PE's, 16 mem., network switch	product '81	MIMD, packet switch. max. 50 simultaneous parallel processes

special algorithms. The communication delay in a simple square array scales as the square root of N .

– Communication and synchronization?

Circuit switching and packet switching represent one broad division of the ways in which the elements of a parallel processor can communicate. Everyday analogies are

the telephone system and the postal system. In circuit switching, a connection is established between the two communicating elements that lasts until the entire message has been transmitted. The lines and switches being used are unavailable to anyone else in the interim. In a packet-switched network, the message is broken up into packets each bearing a destination address and a source address. A switch element within the multistage network encountering a packet reads enough of the packet's address to send it to the correct switch in the next stage, and then it is ready to handle a new packet. In other words, the network can be pipelined.

The choice between these two modes of communication is influenced by a number of factors, including the system software's ability to establish connection topologies at compile or load time vs. at run time, and a lively debate still exists about this choice. Some other aspects of communication and synchronication are discussed below under "Programmer's view".

(3) "...and cooperate..."

– Grain size?

There is a range of grain sizes into which a problem can be cut for parallel execution, as illustrated in Fig. 3. These are strongly dependent on the application, and in turn strongly influence the solution method, including the architecture and the language. For example, a problem that has been divided into a few large tasks or procedures may be well matched to a parallel computer consisting of a few powerful processors, and can afford the overhead of the ADA 'rendezvous' mechanism to control the interaction of these tasks. However, this combination of architecture and language would be a very poor way to obtain speedup if the problem has been divided into a much larger number of much smaller grains. The choice of which level of parallelism to exploit is still an open field of study.

– SIMD/MIMD?

In view of the above discussion, it should not be totally surprising that SIMD/MIMD debates are sometimes heated. Proponents of MIMD feel that the SIMD approach, with each processor performing the same instruction at a given time (or none et all) is severely constrained thereby and unlikely to achieve high speedup from parallelism over any but the narrowest range of applications. On the other hand, in a MIMD

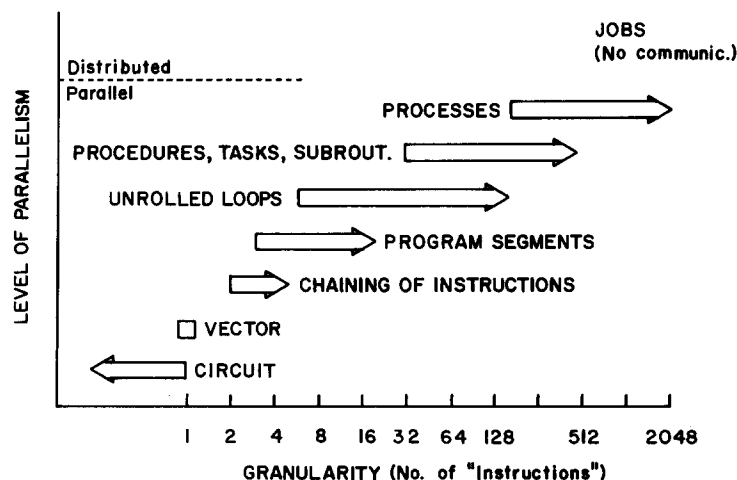


Fig. 3. Levels of parallelism: The granularity or size of the pieces into which the problem is broken down is expressed in numbers of equivalent instructions or statements (after Ostapko [14]).

machine each processor needs enough memory to store its own copy of at least part of the program; this makes designs with more than a few thousand processors seem difficult to achieve. But some researchers are inherently interested in machines with millions of processors, because of the nature of their applications or because of perceived analogies with the brain; SIMD is their only alternative given current technological capabilities. And the general purpose vs. special purpose issue is another unsettled debate.

- Operating system

The CM*, TRAC, ULTRA, and REDIFLOW projects are associated with the problem of an operating system for highly parallel processors. The latter two take rather different approaches to the scheduling of tasks: ULTRA uses a central queue and the 'Fetch-and-Add' synchronization primitive to prevent that queue from becoming a bottleneck, while REDIFLOW uses a distributed control scheme and a pressure-like mechanism to balance the load among processors. Again, an open area of research.

(4) "...To solve big problems fast."

- Which applications?

Which applications will provide enough parallelism to keep all these processors busy, or, to put it more succinctly, 'where's the parallelism?' The list below, compiled by Greg Pfister [15], is a partial answer.

- Large scientific and engineering calculations
 - flow dynamics,
 - particle behavior,
 - weather codes,
 - seismic modeling.

Analysis at NYU has shown that levels of parallelism in the thousands are available from the first three of these. The Cosmic Cube project at Caltech [6] and the users of the DAP machine at Queen Mary College [11] have both analytical and experimental results. The 576-processor GF-11 at IBM Yorktown [2] is being designed for a problem in this area.

- VLSI design automation
 - logic simulation,
 - circuit simulation,
 - shape checking,
 - placement, wiring,
 - logic synthesis (silicon compilation),
 - test generation, fault simulation, timing analysis, logic equivalency analysis.

The YSE experience demonstrates that at least 256-way parallelism is available in logic simulation, and strongly suggests [16] that the degree of available parallelism is in the 1000's for this application as well as for circuit simulation, shape checking, and wiring (the WRM demonstrated 64-way parallelism in chip wiring). Logic synthesis, with a higher degree of interdependencies, is felt to offer parallelism in the 100's.

- Database operations

Both inter- and intra-transaction parallelism are being explored. NON-VON is one project particularly interested in this application.

- Near-term artificial intelligence

The search procedures employed in 'Expert' (Production) Systems and various logic programming applications such as those of the Fifth Generation project are expected to yield on the order of thousand-fold parallelism. DADO and REDIFLOW are two projects aimed at this application.

- Long-term artificial intelligence
 - perception/planning systems,
 - visual recognition,
 - natural language processing,
 - motor control (robotics),
 - learning.

These are all problems that appear suitable for a highly parallel computational model called 'Computing with Connections'; parallelism on the order of tens of thousands appears to be available.

- Computational models?

It is hard not to mix apples and oranges in discussing computational models. Many workers divide parallel processors into *shared-memory* designs (in which a single global address space is equally accessible to all processors) and *message-passing* machines, with a further subdivision of message-passers according to the size of the message (Fahlman [4]). ULTRA and CEDAR are the only pure shared-memory designs in this survey; RP3 is specially designed to function efficiently in both shared memory and message passing configurations, as well as mixtures of both.

However, this addresses only the data mechanism; there is also a control mechanism to be considered. Treleaven [20] breaks these down into control-driven, pattern-driven, demand-driven, and data-driven approaches, representing progressively less explicit control. This leads to the classification in Fig. 4, which shows the relationship of most computational models under discussion. But there are still models that don't quite fit within this classification. One example is Computing with Connections, which treats stored information as a pattern of connections rather than as 'bits on a shelf', a built-in assumption in Fig. 4.

- Programmer's view?

At least three different views of the programmer's role is represented among the parallel processing projects surveyed here:

- (a) At one extreme is the 'dusty FORTRAN deck' approach of the CEDAR project, which relies heavily on a compiler (Parafrase) which automatically converts existing programs for parallel execution and allows the programmer to continue using FORTRAN.
- (b) On the other extreme, the programmer would use an entirely new language in the approach advocated by Backus and other supporters of Functional Programming,

	DATA MECHANISM	
	SHARED MEMORY	MESSAGE PASSING
CONTROL MECHANISM		
control driven	von Neumann	communic. procs.
pattern driven	logic	actors
demand driven	graph reduction	string reduction
data driven	dataflow	

↓
 less
 explicit
 control

Fig. 4. Classification of computational models (after Treleaven [20] and Ostapko [14]).

and this has some influence on the Cellular Computer, REDIFLOW, and Dataflow projects.

- (c) Somewhere in the middle is the performance-starved or budget-limit physicist, say, who is willing to re-code his algorithms for a machine like the Cosmic Cube or to insert some new procedure calls into his program for the GF-11.

Program decomposition and synchronization are performed automatically in the first two of these approaches, and the programmer need not be aware of them. In the last approach, however, some version of the constructs listed below will be needed.

- Program decomposition?

Some of the mechanisms for specifying concurrent execution are:

- Co-routines (Simula),
- Fork & Join (Unix, PL/1, Concurrent Pascal),
- Co-begin (Algo168, CSP),
- Process declaration (ADA).

- Language features?

Several different methods of process interaction are supported by existing language features. These are:

- Remote procedure calls (ADA);
- Sending and receiving messages (CSP, PLITS). This approach is somewhat self-synchronizing;
- Shared variables (Concurrent Pascal). A variety of *synchronization primitives* exists for this approach:
 - busy-waiting (set & test, test, test, ...),
 - semaphores (Dijkstra's P & V),
 - conditional critical regions,
 - monitors,
 - Fetch & Add.

Algorithms?

A fair amount of analysis has been published on parallel algorithms, and the variations in speedup that can be obtained on the same problem certainly show the importance of choosing the right algorithm. Given N processors, the speedup can range all the way from a discouraging $\log N$ up to N itself, although the still-acceptable $N/\log N$ is frequently the best that one can do. Most of the analysis is for SIMD (Sameh [18], Heller [10], Gottlieb and Kruskal [9]). MIMD algorithms are discussed in an excellent paper by Kung [13]. A short quote:

"A parallel algorithm is viewed as a collection of concurrent processes. To ensure that the algorithm works correctly and uses the parallelism effectively, processes must communicate with each other. However, ... the speed of a process is unpredictable. Thus, one can never be sure that an input needed by one process will be produced in time by another process. There are two approaches to solving this problem. The first one is to synchronize processes so that they wait for inputs whenever necessary. This results in a *synchronized parallel algorithm*. The second is to let processes continue or terminate according to the information currently contained in some global variable, so processes never wait for inputs. This results in an *asynchronous parallel algorithm*.

Synchronized algorithms should be used when the fluctuations in process speed are small and when there are only a few processes to be synchronized. They are easier to analyze than asynchronous algorithms. However, the latter are generally more efficient and reliable because they are less likely to get blocked. A tradeoff among the basic processing time, blocking time and synchronization time of a process determines whether it is advantageous to create the maximum possible number of processes.

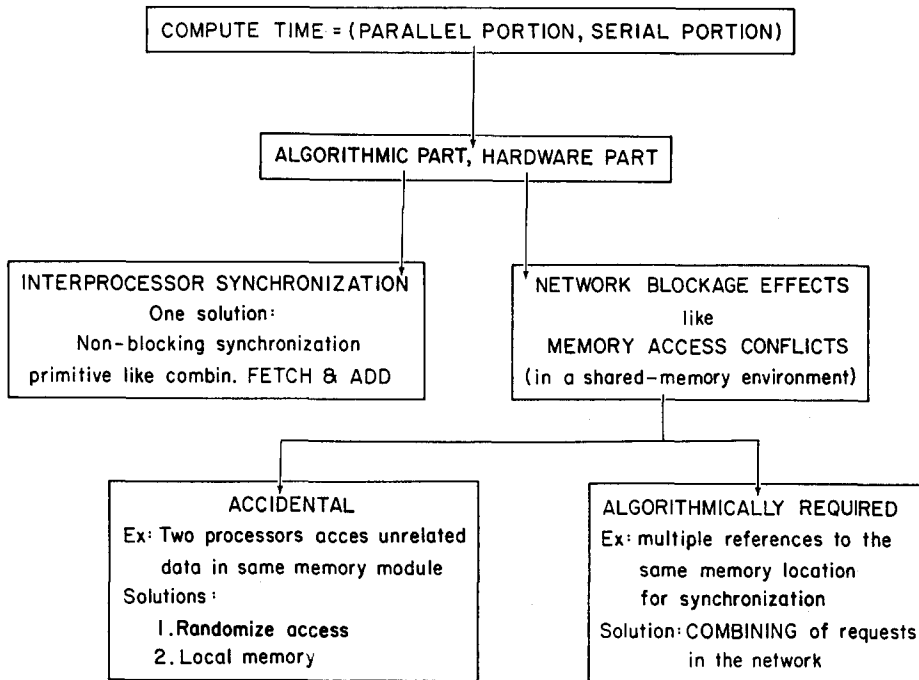


Fig. 5. The serial portion of a parallel program's compute time: The program's speedup due to parallelism is limited by the serial portion, which is influenced both by the algorithm and the hardware design. This example shows the contributions in a shared memory environment, and lists the set of solutions used by the NYU Ultracomputer design to minimize the serial portion.

– How much speedup?

As important as the algorithm is, it is only one factor that determines the actual program's running time. As Amdahl originally noted, the compute time can be divided into the *parallel portion* and the *serial portion*, and no matter how high the degree of parallelism in the former, the speedup will be asymptotically limited by the latter. Pfister [15] subdivides the serial portion into an *algorithmic* part, which refers to serialization intrinsic to the algorithm, and a *hardware-induced* part, which refers to serialization not required by the algorithm but induced by the hardware. Figure 5 shows these relationships in a diagrammatic way for a shared memory computational model; also listed is one set of solutions, namely, those used for the NYU Ultracomputer as well as for RP3 [17]. However, these observations are quite general, and all computational models must supply some such set of solutions. A speedup of 1000 demands that the serial portion be less than 0.1%. Evidence that this is achievable comes from the Caltech Cosmic Cube and NYU Ultracomputer projects.

– Reliability, availability, serviceability?

A critical issue. Some work has been done on redundancy in shuffle-type networks to provide alternate paths in case of defective switches or processors, but much more is needed.

6. Conclusions

A number of worthwhile ideas have been formulated and implemented to some extent. Substantial further progress at this point requires *results* from *real* programs. Performance of

these critical experiments will require the appearance of some substantial new simulation or hardware facilities.

References

- [1] J. Backus, Function-level programming, *IEEE Spectrum* (1982) 22–27.
- [2] J. Beitem, M. Denneau and D. Weingarten, The GF-11 supercomputer, presented at the *12th International Symposium on Computer Architecture*, Boston, June 17–19, 1985.
- [3] J.C. Browne, Understanding execution behavior of software systems, *Computer* **17** (1984) 83–87.
- [4] S.E. Fahlman, Three flavors of parallelism, *Proc. AAAI-80* (1980).
- [5] M.J. Flynn, Some computer organizations and their effectiveness, *IEEE Trans. Comput.* **21** (1972) 948–960.
- [6] G.C. Fox, Concurrent processing for scientific calculations, *Digest of papers, COMPON'84* (IEEE, 1984) 70–73.
- [7] P.O. Fredrickson, R.E. Jones and B.T. Smith, Synchronization and control of parallel algorithms, *Parallel Computing* **2** (1985) 255–264, this issue.
- [8] C. Ghezzi, Concurrency in programming languages: A survey, *Parallel Computing* **2** (1985) 229–241, this issue.
- [9] A. Gottlieb and C.P. Kruskal, Complexity results for permuting data and other computations on parallel processors, *J. ACM* **31** (1984) 193–209.
- [10] D. Heller, A survey of parallel algorithms in numerical linear algebra, *SIAM Rev.* **20** (1978) 740–777.
- [11] J. Howlett, D. Parkinson and J. Sylwestrowicz, DAP in action, *ICL Tech. J.* (May 1983) 330–344.
- [12] P.M. Kogge, Function-based computing and parallelism: a review, *Parallel Computing* **2** (1985) 243–253, this issue.
- [13] H.T. Kung, Synchronized and asynchronous parallel algorithms for multiprocessors, in: *Algorithms and Complexity* (Academic Press, New York, 1976) 153–200. Reprinted in: R. Kuhn and D. Padua, *Tutorial on Parallel Processing* (IEEE, 1981).
- [14] D.L., Ostapko, private communication.
- [15] G.F. Pfister, private communication
- [16] G.F. Pfister, Parallel processing, special-purpose hardware, and design automation, *ACM Computer Science Conference Proceedings* (1983).
- [17] G.F. Pfister, W.C. Brantley, D.A. George, S.L. Harvey, W.J. Kleinfelder, K.P. McAuliffe, E.A. Melton, V.A. Norton and J. Weiss, The IBM research parallel processor prototype (RP3), *1985 International Conference on Parallel Processing* (1985), to appear.
- [18] A. Sameh, Numerical parallel algorithms—A survey, in: *High Speed Computers and Algorithm Organization* (Academic Press, New York, 1977) 207–228. Reprinted in R. Kuhn and D. Padua, *Tutorial on Parallel Processing* (IEEE, 1981).
- [19] J. Schwartz, private communication.
- [20] P. Treleaven, Control-driven, data-driven, and demand-driven computer architecture (extended abstract), *Parallel Computing* **2** (1985) 287–288, this issue.